

2026-06-10-SP7-harness-design

SP7 — Local Harness Design: ddos / scaling / ux / ui (read-only design)

Field	Value
Revision	1
Created	2026-06-10
Last modified	2026-06-10
Status	draft
Scope	helix_code/ (inner Go app dev.helix.code) test surface — design only, no code written
Authority	SP7 plan docs/superpowers/specs/plans/2026-06-10-SP7-testing-qa-plan.md §3 Phase A (Tasks A1–A5) + coverage matrix docs/testing/coverage-matrix.md §2 (GAP-1..GAP-4)
Pattern mirrored	helix_code/tests/stresschaos/{stresschaos.go,chaos.go} + meta-test helix_code/tests/stresschaos/stresschaos_meta_test.go (§1.1 paired mutation)
Evidence rule	every claim cites a real file:line/dir/. Honest gaps marked explicitly (§11.4.6). PLANNING only — nothing executed, no code edited.

Anti-bluff frame. Each harness below is a HelixCode-LOCAL Go harness that produces real captured evidence (CONST-050(B)) — NOT a delegation to a HelixQA shell script. The matter at issue is the delegation-bluff risk flagged in the coverage matrix: ddos/scaling/ux exist today ONLY as HelixQA challenge scripts (docs/testing/coverage-matrix.md:53,54,60), ui is THIN-local (coverage-matrix.md:59). Each harness mirrors the proven stresschaos contract: a Go harness that WRITES then RE-READS a non-empty evidence artefact under qa-results/<run-id>/ (the writeJSON → verifyArtefact pattern at helix_code/tests/stresschaos/stresschaos.go:159-181), plus a paired §1.1 meta-test that plants a defect and asserts the harness DETECTS it (the failTB capturing-TB pattern at helix_code/tests/stresschaos/stresschaos_meta_test.go:21-68).

Table of contents

1. [Verified ground truth](#)
 2. [Harness 1 — ddos \(GAP-1\)](#)
 3. [Harness 2 — scaling \(GAP-2\)](#)
 4. [Harness 3 — ux \(GAP-4\)](#)
 5. [Harness 4 — ui \(GAP-3\)](#)
 6. [Anti-delegation gates \(Task A5\)](#)
 7. [Summary: 4 designs + automatable-vs-operator-attended + gate names](#)
-

1. Verified ground truth

Confirmed by direct inspection this session (read-only):

- **The stresschaos harness is real and is the canonical extend-precedent.** `helix_code/tests/stresschaos/stresschaos.go:159-181` writes JSON evidence then RE-READS it and fails on empty (`verifyArtefact`); :221-295
`RunSustainedLoad` captures p50/p95/p99 to `latency.json` and enforces the §11.4.85 `MinSustainedN=100` floor (:39, :229-231); :315-411
`RunConcurrent` enforces `MinParallelism=10` (:46), guards deadlock by timeout (:363-368), measures goroutine-leak delta (:400-403). Chaos taxonomy Recovered/Degraded/Fatal at `chaos.go:18-41`.
- **The §1.1 meta-test pattern is real.** `stresschaos_meta_test.go:21-68` (`failTB/runWithFailTB`), :72-78 (`isolatedEvidence` redirects `STRESSCHAOS_EVIDENCE_ROOT` to a `t.TempDir()`), :82-133 plant deadlock/leak/error-rate/below-floor, :178-193 proves the happy path writes a non-empty `latency.json`.
- **A REAL booted-server httptest harness ALREADY EXISTS to reuse for ddos.**
`helix_code/internal/server/server_chaos_test.go:74-79`
`newRealServerHarness(t)` builds the REAL Server (real PG pool + real Redis) and serves `srv.router` over `httptest.NewServer`;
`server_stress_test.go:1-2` is `//go:build integration` and drives sustained/concurrent HTTP load via `stresschaos.RunSustainedLoad/RunConcurrent` against `/health` (`server_stress_test.go:37-55`). The ddos harness EXTENDS this — it does not re-invent it.
- **Honest finding (load-bearing for ddos design): there is NO rate-limit middleware today.** `internal/server/server.go:58-61` wires only `gin.Logger` → `gin.Recovery` → `CORSMiddleware` → `SecurityMiddleware`; the only middleware funcs are `authMiddleware` (`server.go:394`), `CORSMiddleware` (:457), `SecurityMiddleware` (:474). `grep` for `tollbooth/ulule/limiter/golang.org/x/time/rate/RateLimit` in `internal/server/*.go` (non-test) returns nothing. The only `rate_limit` token is a *model VerificationStatus* string at `handlers.go:1242-1243`, unrelated to HTTP rate-limiting. **Implication:** today the ddos harness must assert GRACEFUL DEGRADATION (no goroutine leak, no deadlock, no 5xx storm, bounded latency growth) — NOT a 429 refusal ratio, because there is no limiter to produce 429s. A 429-refusal assertion only becomes valid once a real limiter lands (SP-owned). This is documented below, not faked.
- **Worker pool is real and in-process testable for scaling.** `internal/worker/worker_pool.go:195` `WorkerPool`; :205 `NewWorkerPool`; :215 `RegisterWorker`; :259 `AssignTask`; :286-318 `GetPoolStats` returns `total_workers/available_workers/busy_workers/utilization_rate`.

- Existing `internal/worker/worker_pool_stress_test.go` + `worker_pool_chaos_test.go` already use the stresschaos harness against this pool.
- **CLI journey surface is real for ux.** `cmd/cli/main.go:1356` `handleListModels, :1562` `handleGenerate, :2004` `handleCommand` (executes locally via `os/exec` per BLUFF-003). These are the real end-user CLI steps a ux journey drives.
 - **TUI surface is real for ui (note: applications/terminal_ui/, snake_case — NOT terminal-ui).** `applications/terminal_ui/components.go:6` `imports github.com/rivo/tview; :29` `CreateForm, :95` `CreateList, :109-125` `CreateTable (SetCell).` `applications/terminal_ui/main.go` builds the `tview`. `Application`. `tview` is built on `tcell`, which ships `tcell.SimulationScreen` for headless cell-level assertions.
 - **i18n seams are real for ux (CONST-046 resolution invariant).** `internal/server/i18n_seam.go:8-40` `SetTranslator/NoopTranslator` (echoes the message ID verbatim — loud failure); `bundle internal/server/i18n/bundles/active.en.yaml` maps `internal_server_*` IDs → English; `seam test internal/server/i18n_seam_test.go:87` `TestTr_RoutesThroughWiredTranslator`. TUI mirror: `applications/terminal_ui/i18n/translator.go:46-54` `NoopTranslator.T`.
 - **qa-results/ is gitignored** (`helix_code/.gitignore:23`, `root .gitignore:170`) — so raw harness artefacts stay out of git (§11.4.30); only curated evidence is committed at release prep (§11.4.83 `docs/qa/<run-id>/`).
 - **Makefile target pattern is real.** `helix_code/Makefile:114` `stress-chaos, :121` `stress-chaos-meta, :132` `stress-chaos-infra` — the new `test-ddos/test-scaling/test-ux/test-ui + *-meta` targets follow this exact shape (`$(GO_TEST) -race -run ... ; echo evidence under qa-results/`).
 - **CM-gate script + meta-test pattern is real.** `scripts/gates/cross_platform_parity_gate.sh:1-40` (PASS/FAIL/SOFT classification, exit 0/1/2) + `scripts/tests/cross_platform_parity_meta_test.sh` (paired mutation) is the template every `CM- *-HITS-HELIXCODE` gate replicates.
-

2. Harness 1 — ddos (GAP-1)

Owning SP: SP1 (`/api/v1/llm/generate + rate-limit`), SP2 (exposure-endpoint flood).
Fully automatable.

2.1 Test file path + structure

- **Harness:** `helix_code/tests/ddos/ddos_harness.go` — package `ddos`. Thin layer over the existing `stresschaos` primitives + a real booted server. It does NOT duplicate `RunSustainedLoad/RunConcurrent` (§11.4.74 `extend-don't-reimplement`); it adds a `FloodReport` shape that records per-status-code counts and a refusal ratio on top of the `stresschaos` latency capture.
- **Driver test:** `helix_code/tests/ddos/ddos_flood_test.go` (`//go:build integration` — boots a real server like `server_stress_test.go:1`).
- **Meta-test:** `helix_code/tests/ddos/ddos_meta_test.go` (NO build tag, like `stresschaos_meta_test.go` — runs in the fast lane).
- **Makefile:** `test-ddos` (integration-tagged, real server) + `test-ddos-meta` (fast).

2.2 Real assertion + evidence captured

The harness boots the REAL `internal/server.Server` over `httptest.NewServer(srv.router)` — reusing the `newRealServerHarness` construction proven at `server_chaos_test.go:79` — then drives a concurrent flood of $N \geq 100$ (sustained) / $\text{parallelism} \geq 10$ (concurrent) real HTTP requests across a TCP socket against `/health`, `/api/v1/...` and (when SP1 lands) `/api/v1/llm/generate`.

Captured to `qa-results/<run-id>/ddos_<scenario>/:-latency.json` — `p50/p95/p99/min/max + error-rate`, written+re-read non-empty via the `stresschaos.writeJSON/verifyArtefact` path (so a hollow PASS is impossible). - `concurrency_report.json` — `goroutine-before/after delta + deadlock flag` from `RunConcurrent`. - `flood_report.json` (new shape) — `{ requests_sent, status_2xx, status_4xx, status_429, status_5xx, refusal_ratio, p99_under_flood_ms }`.

Assertions (today, no-limiter reality — graceful degradation): 1. **No goroutine leak / no deadlock** under the flood (`RunConcurrent` already fails on `GoroutineDelta > tolerance` and on timeout — `stresschaos.go:400-403, 397-399`). 2. **Zero 5xx** — `status_5xx == 0`. A server-error storm under load is a defect. 3. **Bounded latency growth** — `p99_under_flood_ms ≤ a calibrated multiple of the idle p99 baseline` (threshold calibrated on the project's own first run per §11.4.6, NOT a literature constant per §11.4.107(13)). 4. **Real responses came back over the wire** — body markers asserted exactly as `server_stress_test.go:51-53` asserts the `/health` body contains `healthy` (proves the server actually served, not a no-op).

Assertion when a real limiter lands (SP-owned, gated): when `internal/server` gains rate-limit middleware, the harness additionally asserts `status_429 > 0` AND `refusal_ratio` within an expected band AND that 429s carry a `Retry-After` header — i.e. the limiter *refuses* excess load rather than melting. The harness ships this assertion behind a `DDOS_EXPECT_RATELIMIT` env switch that defaults OFF until the limiter exists, so it never asserts a 429 the codebase cannot produce (honest, per §11.4.6).

2.3 Paired §1.1 meta-test mutation

`ddos_meta_test.go`, mirroring `stresschaos_meta_test.go:82-133`, uses an in-process `httptest.Server` with hand-written handlers (no real DB needed — meta-tests isolate the HARNESS, not the system): - **Plant A (5xx-storm server):** a handler that returns 500 for every request → assert the harness FLAGS the 5xx-storm (records `status_5xx > 0` and fails). A harness that PASSES a 500-storm is a bluff. - **Plant B (latency-bomb server):** a handler that `time.Sleep`s a large fixed delay → assert the harness records a p99 blowout and fails the bounded-latency assertion. - **Plant C (leak server):** a handler that spawns a never-returning goroutine per request → assert `RunConcurrent` flags the goroutine leak (re-using the proven leak detection at `stresschaos.go:400-403`). - **Plant D (limiter-mode, off-by-default):** with `DDOS_EXPECT_RATELIMIT=1` against a server that NEVER returns 429 → assert the harness fails “expected refusals, got none” (so the limiter assertion itself cannot bluff once enabled). - **Positive path:** a 200-OK handler → assert `flood_report.json` is written non-empty (the `TestMeta_PositivePathWritesEvidence` analogue at `stresschaos_meta_test.go:178-193`).

2.4 Anti-delegation gate

CM-DDOS-HITS-HELIXCODE — see §6. Asserts the `ddos run` (local harness OR the HelixQA `ddos_health_flood_challenge.sh`) emitted a `flood_report.json` referencing a LIVE HelixCode endpoint with real p50/p95/p99 — not a config-only PASS.

3. Harness 2 — scaling (GAP-2)

Owning SP: SP5 (SSH worker-pool horizontal scale-out), SP1 (provider-fanout throughput vs N). **Fully automatable.**

3.1 Test file path + structure

- **Harness:** `helix_code/tests/scaling/scaling_harness.go` — package `scaling`. Reuses the `stresschaos` percentile + `writeJSON/verifyArtefact` evidence path; adds a `ScalingReport` shape and a `RunScaleSweep` helper.
- **Driver test:** `helix_code/tests/scaling/scaling_sweep_test.go` (no build tag for the in-process worker-pool sweep; an `//go:build integration` variant for the real-SSH-worker path once SP5 wires it).
- **Meta-test:** `helix_code/tests/scaling/scaling_meta_test.go`.
- **Makefile:** `test-scaling + test-scaling-meta`.

3.2 Real assertion + evidence captured

`RunScaleSweep` exercises the REAL `internal/worker.WorkerPool` (`worker_pool.go:195`): for each N in {1, 2, 4, 8} it `RegisterWorkers` N in-process workers (`worker_pool.go:215`), then drives a fixed total workload of K tasks through `AssignTask(:259)` from `parallelism≥10` concurrent submitters (reusing `stresschaos.RunConcurrent`), and measures **completed-tasks-per-second** plus p50/p95/p99 per-task latency at each N.

Captured to `qa-results/<run-id>/scaling_<scenario>/-`
`scaling_throughput.json` — `[{ n_workers, total_tasks, throughput_tps, p50_ms, p95_ms, p99_ms, pool_utilization }]`
(utilization read from the real `GetPoolStats` map, `worker_pool.go:286-318`),
written+re-read non-empty. - `concurrency_report.json` per N (deadlock/leak guard from `RunConcurrent`).

Assertions: 1. **Scale-out is real, not flat** — throughput at $N=8 \geq$ a calibrated multiple (e.g. $\geq 1.5\times$, threshold from the project's own first run per §11.4.6) of throughput at $N=1$. A pool that ignores added workers shows flat throughput → FAIL. This is the core “the feature actually scales” claim. 2. **Monotonic non-degradation** — throughput does not *regress* as N grows past the contention point (catches a lock-convoy / scheduler bug). 3. **No deadlock / no goroutine leak** at any N (from `RunConcurrent`). 4. **Pool utilization tracks N** — `utilization_rate` from the real `GetPoolStats` is non-zero while busy and returns to idle after (proves workers were actually used, not bypassed).

Honest boundary (§11.4.6): the in-process sweep proves the *pool's* scale-out logic (assignment, scheduling, utilization). True *horizontal* SSH-worker scale-out (`internal/worker/ssh_pool.go`) needs real remote hosts; that path is `//go:build integration` and SKIPS-with-reason (§11.4.3) when no SSH

workers are configured — never a fake PASS. The in-process sweep is the always-available local proof; the SSH sweep is the operator-/CI-provisioned extension.

3.3 Paired §1.1 meta-test mutation

`scaling_meta_test.go`, mirroring the plant-and-detect pattern: - **Plant A (ignores-added-workers pool)**: a fake pool wrapper that serializes all tasks onto one worker regardless of N → assert the harness detects FLAT throughput across N and FAILS the scale-out assertion. (Direct analogue of the SP7-plan A2 requirement: “plant a pool that ignores added workers → assert harness detects flat throughput”). - **Plant B (degrading pool)**: a wrapper whose throughput *drops* as N grows → assert the monotonic-non-degradation assertion fails. - **Plant C (below-floor parallelism)**: call the sweep with `parallelism < MinParallelism` → assert it’s rejected (reuses `RunConcurrent`’s floor check at `stresschaos.go:322-324`). - **Positive path**: a pool that genuinely scales → `scaling_throughput.json` written non-empty with rising throughput.

3.4 Anti-delegation gate

CM-SCALING-HITS-HELIXCODE — see §6. Asserts the scaling run exercised real `internal/worker` with a captured throughput table referencing live worker registration counts.

4. Harness 3 — ux (GAP-4)

Owning SP: SP4 (end-user CLI journey), SP6 (human-in-loop UX flows). **Mostly automatable** (see honest boundary on subjective UX).

4.1 Honest framing: what “UX” means for an API/CLI-only product (§11.4.6)

HelixCode ships no consumer GUI as its primary surface — it is a server + CLI + (secondary) TUI/desktop. “User experience” for a no-GUI product is NOT subjective aesthetics (which is genuinely operator-attended); it is the set of **mechanically-checkable interaction invariants** that determine whether the product is *usable*:

1. **Journey completeness** — the documented end-user CLI journey (`init` → `list-models` → `generate` → `command-exec`) runs end-to-end and each step produces real, asserted output (not a canned string).
2. **Error-message clarity** — when a step fails, the message is a real, locale-resolved, actionable string — NOT a raw message ID, NOT an empty string, NOT a Go error like `nil pointer`.
3. **i18n resolution (CONST-046)** — user-facing text resolves through a wired translator to locale text, never leaking the raw `internal_server_*` / TUI message ID. This is the single most mechanizable UX invariant in this codebase because the seams already exist (`i18n_seam.go:8-40`, `terminal_ui/i18n/translator.go:46-54`).
4. **Response-shape consistency** — API error responses share a consistent JSON envelope (same top-level keys) so a client can parse them uniformly. Inconsistent shapes are a real UX defect for API consumers.

What is genuinely **operator-attended** and honestly excluded: subjective “does this feel pleasant / is the wording natural” judgement, and visual layout aesthetics. Those are tracked as operator-attended per §11.4.52, never auto-PASSED.

4.2 Test file path + structure

- **Harness/test:** `helix_code/tests/ux/ux_journey_test.go` (//go:build integration for the live-server journey; an in-process variant for the i18n + response-shape invariants needs no infra).
- **Meta-test:** `helix_code/tests/ux/ux_meta_test.go`.
- **Makefile:** `test-ux + test-ux-meta`.

4.3 Real assertion + evidence captured

Drives the real CLI journey programmatically (zero human action after startup, §11.4.98) — invoking the real handlers `handleListModel`s/`handleGenerate`/`handleCommand` (`cmd/cli/main.go:1356,1562,2004`) against a booted server, OR shelling the built `bin/cli` via `os/exec` so the FULL binary path is exercised.

Captured to `docs/qa/<run-id>/ux_journey/` (§11.4.83 — bidirectional transcript, committed at release prep) and `qa-results/<run-id>/ux_<scenario>/`: - `journey_transcript.jsonl` — one line per step: `{ step, command_sent, response_received, assertion, verdict }` (the FULL bidirectional thread, §11.4.83). - `i18n_resolution.json` — `[{ message_id, locale, resolved_text, leaked_id: bool }]`. - `error_shape_report.json` — top-level key sets across sampled error responses + a consistency verdict.

Assertions: 1. **Each journey step asserts REAL output** — e.g. `list-models` output contains real provider/model rows (not a hardcoded list, ties BLUFF-002); `generate` output is real LLM text (ties BLUFF-001); `command-exec` surfaces a real exit code (ties BLUFF-003). A step that prints a canned constant FAILS. 2. **i18n no-leak** — for each sampled user-facing string, `resolved_text != message_id` when a real translator is wired; `leaked_id == false`. Drives a real translator (like `i18n_seam_test.go:87`) and asserts the resolved text differs from the raw ID. A leaked `internal_server_qa_engine_disabled` to a user is a CONST-046 UX defect. 3. **Error clarity** — on a forced failure, the error body is non-empty, is a resolved string, and names the subject ($\geq$ a minimum descriptive length, mirroring the §11.4.91 clarity bar). 4. **Response-shape consistency** — sampled error responses share the same top-level envelope keys.

4.4 Paired §1.1 meta-test mutation

`ux_meta_test.go`: - **Plant A (canned-string step)**: a journey step whose handler returns a fixed constant instead of real output → assert the journey's real-output assertion FAILS the canned plant (the SP7-plan A3 requirement: “plant a step that prints a canned string → assert journey asserts on REAL output and FAILS”). - **Plant B (leaked message ID)**: wire the `NoopTranslator` (which echoes the ID, `i18n_seam.go` default) and assert the `i18n-no-leak` assertion catches `resolved_text == message_id` → FAIL. (This is a true paired mutation: the Noop path IS the planted defect.) - **Plant C (inconsistent error shape)**: feed two error responses with divergent top-level keys → assert the consistency assertion fails. - **Positive path**: a real journey → `journey_transcript.jsonl` written non-empty with per-step real-output verdicts.

4.5 Anti-delegation gate

CM-UX-HITS-HELIXCODE — see §6. Asserts the ux journey drove the real CLI/server with a captured bidirectional transcript referencing live endpoints/binary — not a config-only PASS.

5. Harness 4 — ui (GAP-3)

Owning SP: SP4 (terminal interaction), SP6 (human-in-loop flow-node UI). **Partly automatable** (TUI headless), **partly operator-attended** (desktop GUI + non-introspectable surfaces).

5.1 Honest split: TUI (automatable headless) vs Desktop (operator-attended)

Surface	Tech	Headless-testable?	Verdict
TUI	tview/tcell (terminal_ui/ components.go:6)	YES — tcell.SimulationScreen renders to an in-memory cell grid; inject keys, read back exact cells	Fully automatable
Desktop GUI	Fyne v2.7 (applications/ desktop/)	Partial — Fyne has a test package (fyne.io/fyne/ v2/test) that drives widgets headlessly WITHOUT a display server; pixel-level / native- window behaviour is NOT	Widget-tree automatable; pixel/native = operator- attended
Non- introspectable surfaces (canvas- rendered, blank ally tree)	—	Pixel-oracle (§11.4.117 CV/ OCR) or operator-attended	Operator- attended fallback when neither hierarchy nor pixel oracle works

5.2 Test file path + structure

- **TUI harness/test:** helix_code/tests/ui/tui_interaction_test.go — drives the real tview components via tcell.NewSimulationScreen().
- **Desktop harness/test:** helix_code/tests/ui/desktop_widget_test.go — drives Fyne widgets via fyne.io/fyne/v2/test (build-tagged; SKIP-with-reason when the Fyne test harness or GUI build tag is unavailable, never fake-PASS).
- **Pixel-oracle (§11.4.117) self-validation:** helix_code/tests/ui/ocr_oracle_test.go — only if/when a non-introspectable surface is in scope; carries the golden-good/golden-bad fixture pair.
- **Meta-test:** helix_code/tests/ui/ui_meta_test.go.
- **Makefile:** test-ui + test-ui-meta.

5.3 Real assertion + evidence captured

TUI (automatable): construct the real TUI components (CreateList/CreateTable/CreateForm, components.go:95, 109, 29) bound to a tcell.SimulationScreen of fixed size; inject a key sequence (e.g. arrow-down, enter) via the simulation screen; call Draw; read back the rendered cell grid via SimulationScreen.GetContents() and assert the expected glyphs/text appear at the expected cell coordinates.

Captured to qa-results/<run-id>/ui_<scenario>/: - rendered_cells.json — { width, height, asserted_strings: [{ text, row, col, found }] }
written+re-read non-empty. - tui_key_trace.jsonl — injected keys + resulting screen-

state hash per step. - (pixel-oracle path only) `ocr_self_validation.json` — golden-good PASS + golden-bad FAIL verdicts.

Assertions: 1. **Real rendered content** — the asserted strings appear in the actual rendered cell grid (a TUI that renders nothing → empty grid → FAIL). This is genuine pixel-equivalent evidence: `tcell` actually composited the widgets. 2. **Interaction works** — injecting a navigation key changes the rendered state in the expected way (e.g. selection highlight moves), proving the input path is wired, not stubbed. 3. **i18n in the TUI** — rendered labels are resolved text, not raw message IDs (reuses the `terminal_ui/i18n/translator.go:46` seam; a leaked ID in a rendered cell → FAIL). 4. **(pixel-oracle, §11.4.107(10))** the CV/OCR analyzer PASSES its golden-good fixture and FAILS its golden-bad fixture — an analyzer that PASSES golden-bad is itself a bluff and the gate FAILS.

5.4 Paired §1.1 meta-test mutation

`ui_meta_test.go`: - **Plant A (blank-render component)**: a component whose `Draw` writes nothing to the simulation screen → assert the rendered-content assertion FAILS (empty cell grid is not a PASS). - **Plant B (dead key-handler)**: a component that ignores injected keys → assert the interaction assertion detects the unchanged screen-state and FAILS. - **Plant C (leaked message ID in cell)**: render with `NoopTranslator` → assert a raw message ID in the cell grid is caught → FAIL. - **Plant D (pixel-oracle self-validation)**: feed the OCR analyzer its golden-BAD fixture → assert the analyzer FAILS it (if the analyzer PASSES golden-bad, the meta-test FAILS, per §11.4.107(10)). - **Positive path**: real render → `rendered_cells.json` written non-empty with found-string verdicts.

5.5 Honest gaps (operator-attended, §11.4.52)

- **Desktop pixel-/native-window behaviour** (actual on-screen Fyne pixels, native dialogs, OS clipboard) — NOT headlessly assertable; tracked as operator-attended with a §11.4.52 migration item. Widget-tree state IS automatable via `fyne.io/fyne/v2/test`; only the pixel/native layer is the gap.
- **Subjective visual quality** (spacing, color harmony) — operator-attended, never auto-PASSED.
- **Non-introspectable canvas surfaces** where BOTH the widget tree is blank AND no OCR/CV oracle is feasible — SKIP-with-reason (§11.4.3) + tracked operator-attended item, never fake-PASS.

5.6 Anti-delegation gate

CM-UI-HITS-HELIXCODE — see §6. Asserts the ui run asserted on real rendered cells / OCR self-validation referencing the live TUI/desktop components — not a config-only PASS.

6. Anti-delegation gates (Task A5)

Four pre-build gates convert the delegated HelixQA rows from coverage-bluff risk into evidence-backed delegation. Each follows the `scripts/gates/cross_platform_parity_gate.sh` template (PASS/FAIL/SOFT classification, exit 0/1/2) and ships a paired §1.1 meta-test under `scripts/tests/` (mirroring `scripts/tests/cross_platform_parity_meta_test.sh`).

Gate	Lives at (PLANNED)	Asserts (parses qa-results/)	Paired §1.1 mutati
CM-DDOS-HITS-HELIXCODE	scripts/gates/ddos_hits_helixcode_gate.sh	the ddos run emitted flood_report.json referencing a LIVE HelixCode endpoint with real p50/p95/p99 (non-empty, non-placeholder)	strip the endpoint hit/late assertio from the evidence → gate FAILs
CM-SCALING-HITS-HELIXCODE	scripts/gates/scaling_hits_helixcode_gate.sh	scale-out exercised real internal/worker with a captured scaling_throughput.json table (real worker counts, rising throughput)	replace through table w a flat/ empty → gate FAILs
CM-UX-HITS-HELIXCODE	scripts/gates/ux_hits_helixcode_gate.sh	the journey drove the real CLI/ server with a captured bidirectional journey_transcript.jsonl (both directions present, real output)	one-sid transcri canned output gate FAILs
CM-UI-HITS-HELIXCODE	scripts/gates/ui_hits_helixcode_gate.sh	the TUI/desktop run asserted on real rendered cells (rendered_cells.json) / OCR self-validation passed golden-good + failed golden-bad	empty grid / analyze that PASSE golden-bad → gate FAILs

Each gate is wired into the pre-build sweep (the make/scripts gate runner) and each Makefile harness target writes the evidence the gate parses. The gates are the mechanical proof that a delegated HelixQA challenges/scripts/*.sh last-run actually hit HelixCode — not a config-only PASS (CONST-050(B)).

7. Summary

Four HelixCode-LOCAL harness designs (each: Go harness writing re-read non-empty evidence to qa-results/<run-id>/, plus a paired §1.1 plant-and-detect meta-test, mirroring stresschaos):

#	Harness	File path (PLANNED)	Real assertion (core)	Evidence
1	ddos	helix_code/tests/ddos/ddos_harness.go +	concurrent flood vs REAL booted internal/server (reuses	latency.json, concurrency_rep flood_report.js

#	Harness	File path (PLANNED)	Real assertion (core)	Evidence
		ddos_flood_test.go + ddos_meta_test.go	newRealServerHarness); no goroutine-leak/deadlock, zero 5xx, bounded p99; 429-refusal assertion behind DDOS_EXPECT_RATELIMIT (OFF until a limiter lands — honest: no rate-limit middleware exists today, server.go: 58-61)	
2	scaling	helix_code/tests/ scaling/ scaling_harness.go + scaling_sweep_test.go + scaling_meta_test.go	throughput sweep over REAL internal/ worker.WorkerPool at N=1,2,4,8 with p50/p95/p99; assert genuine scale-out (not flat) + no degradation + real utilization	scaling_throughput concurrency_rep
3	ux	helix_code/tests/ux/ ux_journey_test.go + ux_meta_test.go	real CLI journey (init→list→generate→exec, real-output assertions tying BLUFF-001/002/003) + i18n no-leak (CONST-046) + error clarity + response-shape consistency	journey_transcript (\$11.4.83), i18n_resolution error_shape_rep
4	ui	helix_code/tests/ui/ tui_interaction_test.go + desktop_widget_test.go + ocr_oracle_test.go + ui_meta_test.go	TUI via tcell.SimulationScreen — inject keys, assert real rendered cells + interaction + i18n no-leak; Fyne via fyne.io/fyne/v2/test; pixel-oracle self-validated (\$11.4.107(10))	rendered_cells tui_key_trace. ocr_self_valida

Fully automatable vs operator-attended (honest, §11.4.6): - **Fully automatable:** ddos (entire); scaling (in-process worker-pool sweep); ux (journey + i18n + error-clarity + response-shape); ui (TUI headless via `tcell.SimulationScreen`, Fyne widget-tree via the Fyne test package). - **Operator-attended (genuine gaps, tracked §11.4.52, never fake-PASS):** ux subjective aesthetics/wording-naturalness; ui desktop pixel-/native-window rendering; ui non-introspectable canvas surfaces where neither widget-tree nor an OCR/CV oracle is feasible. - **Conditional-automatable (SKIP-with-reason until provisioned, §11.4.3):** scaling real-SSH horizontal scale-out (needs configured remote workers); ddos 429-refusal assertion (needs a rate-limit middleware that does NOT exist today — `server.go: 58-61`).

The four anti-delegation CM- *-HITS-HELIXCODE gate names (each with a paired §1.1 mutation, templated on scripts/gates/cross_platform_parity_gate.sh): 1. CM-DDOS-HITS-HELIXCODE 2. CM-SCALING-HITS-HELIXCODE 3. CM-UX-HITS-HELIXCODE 4. CM-UI-HITS-HELIXCODE

Sources verified 2026-06-10

- helix_code/tests/stresschaos/stresschaos.go:39,46,159-181,221-295,315-411 (harness primitives, floors, evidence write+re-read).
- helix_code/tests/stresschaos/chaos.go:18-41 (Recovered/Degraded/Fatal taxonomy).
- helix_code/tests/stresschaos/stresschaos_meta_test.go:21-68,72-78,82-133,178-193 (§1.1 failTB plant-and-detect pattern).
- helix_code/internal/server/server.go:47,58-61,394,457,474 (server wiring — NO rate-limit middleware).
- helix_code/internal/server/handlers.go:1242-1243 (rate_limit is a model-status string, not HTTP limiting).
- helix_code/internal/server/server_chaos_test.go:74-79 (newRealServerHarness) + server_stress_test.go:1-2,37-55 (real httptest server load pattern).
- helix_code/internal/worker/worker_pool.go:195,205,215,259,286-318 (real worker pool + GetPoolStats).
- helix_code/cmd/cli/main.go:1356,1562,2004 (real CLI journey handlers).
- helix_code/applications/terminal_ui/components.go:6,29,95,109-125 (real tview TUI) + applications/terminal_ui/i18n/translator.go:46-54.
- helix_code/internal/server/i18n_seam.go:8-40 + internal/server/i18n/bundles/active.en.yaml + internal/server/i18n_seam_test.go:87 (CONST-046 i18n seam).
- helix_code/Makefile:114,121,132 (target pattern) + scripts/gates/cross_platform_parity_gate.sh:1-40 + scripts/tests/cross_platform_parity_meta_test.sh (gate + paired-mutation template).
- helix_code/.gitignore:23,root.gitignore:170 (qa-results/ ignored).